# Bancos de dados relacionais espaciais

Disciplina: Programação aplicada à engenharia cartográfica

Maurício C. M. de Paulo - D.Sc.

1 de abril de 2026

# Conceitos Fundamentais

**Banco de Dados Relacional (BDR)**:

- Dados organizados em tabelas (relações)
- Linhas $\rightarrow$ tuplas
- Colunas $\rightarrow$ atributos
- Chaves primárias e estrangeiras

Modelo baseado em:

- Álgebra relacional
- SQL (Structured Query Language)

Exemplo de SGBD:

- **PostgreSQL database system**
- **MySQL database system**

# SQL — Structured Query Language

**SQL** é a linguagem padrão para manipulação de bancos de dados relacionais.

Principais usos:

- Consultar dados
- Inserir novos registros
- Atualizar registros existentes
- Remover dados
- Definir estrutura de tabelas

Bancos que utilizam SQL:

- PostgreSQL
- MySQL
- SQLite (e Geopackage)
- DuckDB
- SQL Server

# Postgis no Docker

Recuperando nosso container de PostgreSQL+PostGIS da aula passada.

## Iniciando o container

```
docker start postgis
```

## Criando um banco novo

```
docker exec -it postgis createdb geodb -U postgres
```

## Conectando ao banco pelo terminal

```
docker exec -it postgis psql -U postgres -d geodb
```

# Estrutura básica de uma consulta

A operação mais comum em SQL é a consulta de dados usando SELECT.

```
SELECT coluna1, coluna2
FROM tabela
WHERE condicao;
```

Componentes:

- **SELECT** — define as colunas retornadas
- **FROM** — define a tabela consultada
- **WHERE** — filtra os registros

Exemplo:

```
SELECT nome, populacao
FROM cidades
WHERE populacao > 100000;
```

# Inserindo, atualizando e removendo dados

SQL também permite modificar os dados.

Inserir registros:

```
INSERT INTO cidades (nome, populacao)
VALUES ('Campinas', 1214000);
```

Atualizar registros:

```
UPDATE cidades
SET populacao = 1300000
WHERE nome = 'Campinas';
```

Remover registros:

```
DELETE FROM cidades
WHERE nome = 'Campinas';
```

# Operações de agregação

Funções de agregação resumem dados.

```
SELECT COUNT(*)
FROM cidades;
```

Funções comuns:

- `COUNT()` — número de registros
- `SUM()` — soma
- `AVG()` — média
- `MIN()` — valor mínimo
- `MAX()` — valor máximo

Exemplo:

```
SELECT AVG(populacao)
FROM cidades;
```

# Relacionando tabelas — JOIN

Bancos relacionais permitem combinar dados de várias tabelas.
Exemplo de duas tabelas:

- cidades
- estados

Consulta com **JOIN**:

```
SELECT c.nome, e.nome
FROM cidades c
JOIN estados e
ON c.estado_id = e.id;
```

**JOIN** permite:

- combinar informações
- evitar duplicação de dados
- modelar relações entre entidades

# Tabelas em bancos relacionais

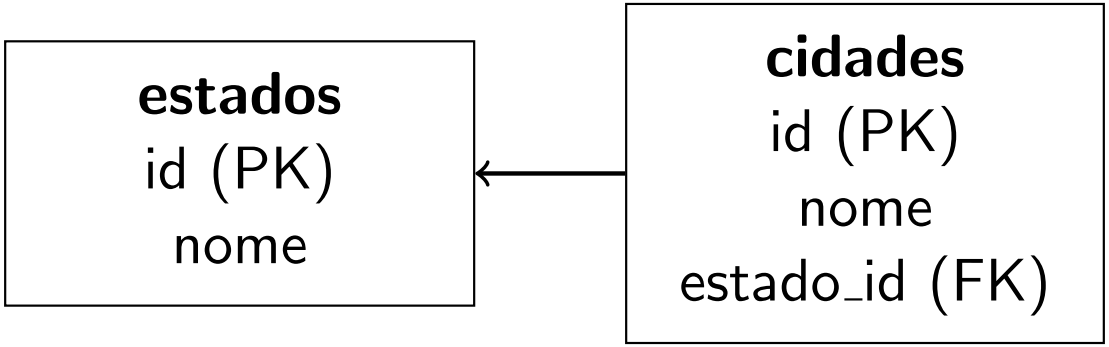
Dados em bancos relacionais são organizados em **tabelas**.

| **id** | **nome** | **população** |
|---|---|---|
| 1 | Curitiba | 1774000 |
| 2 | Campinas | 1214000 |
| 3 | Recife | 1490000 |
| | | |

- **Colunas** representam atributos
- **Linhas** representam registros
- Cada tabela normalmente possui uma **chave primária (ID)**

# Relacionamento entre tabelas (Modelo ER)

Bancos relacionais conectam tabelas usando **chaves estrangeiras**.




cidade pertence a um estado

Exemplo de consulta combinando tabelas:

```
SELECT c.nome, e.nome
FROM cidades c
JOIN estados e
ON c.estado_id = e.id;
```

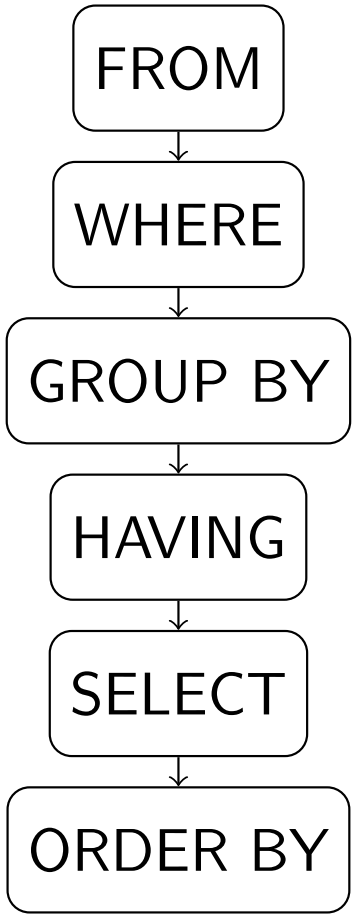
# Ordem de execução de uma consulta SQL

Embora a consulta seja escrita nesta ordem:

```
SELECT nome, populacao
FROM cidades
WHERE populacao > 100000
ORDER BY populacao DESC;
```

- Primeiro o banco identifica a **tabela** (FROM)
- Depois aplica **filtros** (WHERE)
- Em seguida realiza **agrupamentos**
- Só então seleciona as **colunas finais**
- Por último ordena os resultados

O banco executa as operações aproximadamente nesta ordem:



# Extensão Espacial

BDRs podem ser estendidos para armazenar geometrias.

Exemplo:

- **PostGIS** extensão do PostgreSQL

Novo tipo de dado:

- GEOMETRY
- GEOGRAPHY

Permite armazenar:

- Pontos
- Linhas
- Polígonos

E executar consultas espaciais.

# Shapefile vs PostgreSQL/PostGIS vs DuckDB Spatial

| | Shapefile | PostgreSQL + PostGIS | DuckDB + Spatial |
|---|---|---|---|
| Multiusuário | Não | Sim | Limitado |
| Transações ACID | Não | Sim | Sim |
| Cliente–Servidor | Não | Sim | Não |
| Instalação | Simples | Servidor dedicado | Arquivo único |
| Índice Espacial | .qix | GiST / SP-GiST | R-Tree |
| SQL Completo | Não | Sim | Sim |
| Escalabilidade | Baixa | Alta | Média |

**Posicionamento:**

Shapefile → formato legado de troca.

PostgreSQL + PostGIS → infraestrutura robusta multiusuário.

DuckDB Spatial → banco analítico leve e embarcado.

# Modelo Espacial

Cada feição espacial é composta por:

- Atributos (campos tabulares)
- Geometria (coluna espacial)

Estrutura típica:

```
id | nome | area | geom
1 | Rio de Janeiro | 43,696 | MULTIPOLYGON(...)
```

A coluna `geom` armazena objetos como:

- POINT
- LINESTRING
- POLYGON

# Consultas Espaciais

Além de consultas tradicionais:

- SELECT
- JOIN
- GROUP BY

Existem funções espaciais:

- ST_Buffer()
- ST_Intersects()
- ST_Contains()
- ST_Area()

Exemplo conceitual:

Selecionar municípios que intersectam um rio.

# PostgreSQL + PostGIS — Visão Geral

**PostgreSQL**:

- SGBD relacional cliente–servidor
- ACID completo
- Controle de concorrência (MVCC)
- Extensível

**PostGIS**:

- Extensão espacial do PostgreSQL
- Adiciona tipos GEOMETRY e GEOGRAPHY
- Implementa padrões OGC Simple Features

Resultado:

Banco relacional + engine espacial robusta

# Conceitos Fundamentais do PostGIS

1. **Tipo GEOMETRY**
   - Armazena POINT, LINESTRING, POLYGON, etc.
   - Associado a um SRID (sistema de referência)
2. **Funções Espaciais**
3. **Índices Espaciais**
   - GiST (Generalized Search Tree)
   - Filtragem por bounding box
   - Otimização de ST_Intersects, ST_Within, etc.
4. **Modelo Cliente–Servidor**
   - Multiusuário
   - Controle transacional
   - Ideal para produção

# Índices Espaciais

Consultas espaciais são custosas.

Solução: Índices espaciais.

- R-Tree
- GiST (PostgreSQL)

Benefícios:

- Redução drástica de tempo de consulta
- Filtragem por bounding box

Sem índice $\rightarrow$ varredura completa da tabela.

# Índices Espaciais: Por que são importantes?

Problemas comuns em consultas espaciais:

- Encontrar objetos que **intersectam** uma área
- Selecionar feições **próximas**
- Identificar objetos dentro de um **bounding box**

**Sem índice espacial:**

- Cada geometria precisa ser testada
- Complexidade aproximada: $O(n)$

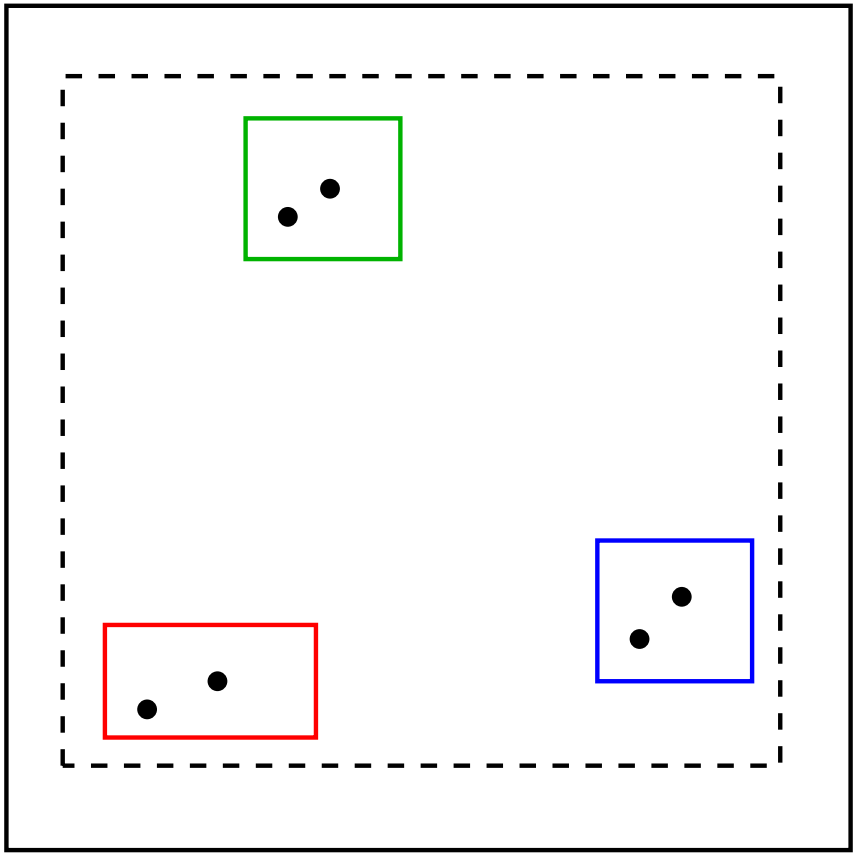
**Com índice espacial:**

- Estrutura hierárquica de envelopes
- Redução drástica do número de testes geométricos
- Complexidade aproximada: $O(\log n)$

**Ideia central:**

- Primeiro filtrar por **bounding boxes**
- Depois aplicar a operação geométrica precisa

# Índices espaciais: Estrutura de um R-tree



Objetos agrupados em **Minimum Bounding Rectangles (MBR)**

- Cada nó armazena o **retângulo mínimo** que cobre seus filhos
- Consultas percorrem apenas ramos relevantes
- Estrutura semelhante a uma árvore B para dados espaciais

# Índices Espaciais em Formatos Vetoriais

| Formato | Tipo de índice | Observação |
|---|---|---|
| Shapefile | Arquivo .qix ou .sbn/.sbx | Índice externo baseado em R-tree |
| GeoPackage | R-tree interno (SQLite) | Tabelas auxiliares automáticas |
| PostGIS | GiST / SP-GiST | Índices espaciais robustos para grandes bases |

**Exemplo em PostGIS:**

```
CREATE INDEX idx_geom
ON cidades
USING GIST (geom);
```

**Benefícios:**

- Consultas espaciais muito mais rápidas
- Fundamental para bases grandes
- Usado automaticamente em operações como: `ST_Intersects`, `ST_DWithin`, `ST_Contains`

# Consultas Espaciais — Exemplos SQL (PostGIS)

## 1 Criar tabela espacial

```
CREATE TABLE municipios (
id SERIAL PRIMARY KEY,
nome TEXT,
populacao INTEGER,
geom GEOMETRY(MULTIPOLYGON, 4674)
);
```

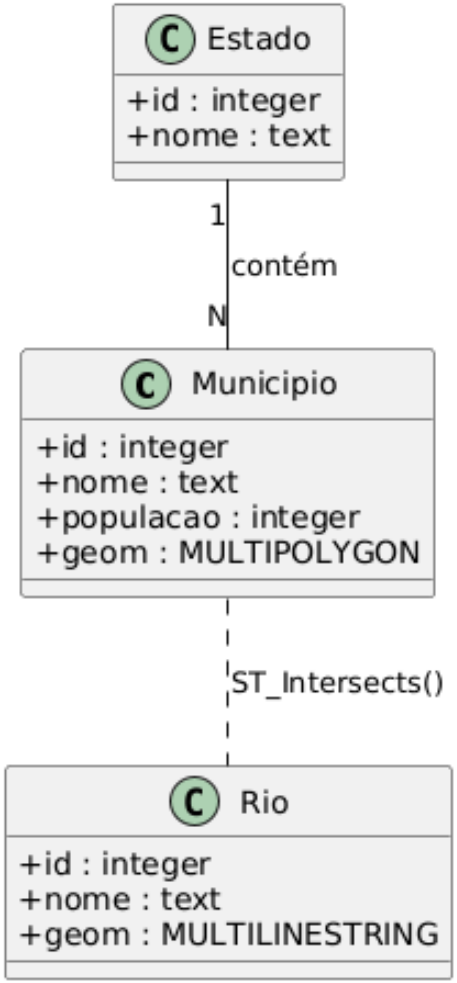
## 2 Criar índice espacial

```
CREATE INDEX idx_municipios_geom
ON municipios
USING GIST (geom);
```

# Exercícios PostGIS

1. Instalar o PostGIS (Docker recomendado)
2. Criar um banco de dados
3. Acessar pelo QGIS
4. Abrir o DB Manager.
5. Criar uma tabela.
6. Importar um ShapeFile para o PostGIS.

# Modelo ER Espacial



# Consultas Espaciais — Exemplos SQL

## 3 Municípios que intersectam um rio

```sql
SELECT m.nome
FROM municipios m
JOIN rios r
ON ST_Intersects(m.geom, r.geom)
WHERE r.nome = 'Rio X';
```

## 4 Área em km²

```sql
SELECT nome,
ST_Area(geom) / 1000000 AS area_km2
FROM municipios;
```

# O que é DuckDB?

**DuckDB** é um banco de dados analítico embutido (embedded OLAP).

Características principais:

- Arquivo único (.duckdb)
- Sem servidor (in-process)
- Execução vetorizada (vectorized engine)
- Otimizado para leitura intensiva (OLAP)

Casos de uso típicos:

- Análise local de grandes datasets
- Ciência de dados
- Processamento de Parquet
- Integração com Python / R

# DuckDB: Conceitos Arquiteturais Fundamentais

1. **Embedded Database**
   - Executa dentro do processo Python
   - Não há daemon ou serviço separado
2. **Columnar Storage**
   - Armazenamento orientado a colunas
   - Ideal para agregações e scans analíticos
3. **Integração com formatos modernos**
   - Parquet (leitura direta)
   - CSV
   - Arrow
   - Extensão Spatial (tipos GEOMETRY)
4. **OLAP vs OLTP**
   - Excelente para consultas analíticas
   - Não projetado para alta concorrência transacional

# Instalando DuckDB CLI com Pixi

O **Pixi** permite criar ambientes isolados e instalar ferramentas rapidamente.

## 1. Criar um novo projeto Pixi

```
pixi init pixi_env
cd pixi_env
```

## 2. Instalar o DuckDB CLI

```
pixi add duckdb-cli
```

## 3. Executar o DuckDB

```
pixi run duckdb
```

## Verificando a instalação

```
SELECT version();
```

- Ambiente reproduzível
- Sem necessidade de instalar o banco globalmente
- Ideal para aulas e projetos de análise de dados

# Usando DuckDB para consultar dados de cidades

**DuckDB** permite consultar dados diretamente de arquivos online usando SQL.

Exemplo usando um dataset público de cidades:

```
INSTALL httpfs;
LOAD httpfs;

CREATE TABLE cidades AS
SELECT *
FROM read_csv_auto(
'https://raw.githubusercontent.com/datasets/world-cities/master/data/world-cities.c
);
```

Agora podemos executar consultas SQL normalmente:

```
SELECT name, country
FROM cidades
WHERE country = 'Brazil'
LIMIT 10;
```

# Exemplo — DuckDB + Spatial em Python

**Instalação (uma vez):**

```
pixi add duckdb
```

**Exemplo em Python (parte 1):**

```python
import duckdb

# conecta (arquivo será criado se não existir)
con = duckdb.connect("dados.duckdb")

# instala e carrega extensão espacial
con.execute("INSTALL spatial;")
con.execute("LOAD spatial;")
```

# Exemplo — DuckDB + Spatial em Python

```python
# cria tabela espacial
con.execute("""
    CREATE TABLE municipios AS
    SELECT
        1 AS id,
        'Municipio A' AS nome,
        ST_GeomFromText(
            'POLYGON((-48 -15, -48 -16, -47 -16, -47 -15, -48 -15))'
        ) AS geom;
""")

# consulta área
result = con.execute("""
    SELECT nome,
    ST_Area(geom) AS area
    FROM municipios;
""").fetchall()

print(result)
```

# Exemplo — Download de Shapefile + DuckDB Spatial

```
import requests, zipfile, io
import duckdb

# 1 Download (Natural Earth - países 110m, leve)
url = "https://naturalearth.s3.amazonaws.com/110m_cultural/ne_110m_admin_0_countrie
response = requests.get(url)
response.raise_for_status()

# 2 Extrai zip em memória
with zipfile.ZipFile(io.BytesIO(response.content)) as z:
    z.extractall("ne_countries")

# 3 Conecta ao DuckDB
con = duckdb.connect("world.duckdb")
con.execute("INSTALL spatial;")
con.execute("LOAD spatial;")
```

# Exemplo — Download de Shapefile + DuckDB Spatial

```
# 4 Lê shapefile direto via GDAL
con.execute("""
    CREATE TABLE countries AS
    SELECT * FROM ST_Read(
    'ne_countries/ne_110m_admin_0_countries.shp');
""")

# 5 Calcula área (m² | CRS 4326->3857)
result = con.execute("""
    SELECT NAME,
    ST_Area(ST_Transform(geom, 3857)) AS area
    FROM countries
    ORDER BY area DESC
    LIMIT 5;
""").fetchall()

for row in result:
    print(row)
```